INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Análisis y Diseño de Algoritmos

Práctica 4. Algoritmos Voraces y Codificación Huffman

Gonzalez Ramirez Ángeles Nataly

Melo Cruz Leonardo Galdino

Olvera Gil Leonel

3BV1

10 de noviembre de 2026

1. Objetivo

Implementar un compresor y descompresor de Huffman eficiente (empaquetado de bits real con `BitWriter` / `BitReader`), analizar teóricamente el algoritmo sobre un mensaje de 36 caracteres, auditar los errores típicos que produce una IA ante prompts ingenuos (cadenas de texto binarias y bug del último byte) y evaluar el compresor con distintos tipos de archivos.

2. Fase 1. Teoría

Mensaje: **M = "ABRACADABRA_ALGORITMO_AVANZADO_ESCOM"**, con N = 36 caracteres y k = 17 símbolos únicos.

2.1 Tabla de frecuencias (inciso A)

Símbolo	f_i	p_i = f_i/36	−p_i·log2(p_i)
A	9	0.2500	0.5000
O	4	0.1111	0.3522
R	3	0.0833	0.2987
_	3	0.0833	0.2987
B	2	0.0556	0.2317

Símbolo	f_i	p_i = f_i/36	−p_i·log2(p_i)
C	2	0.0556	0.2317
D	2	0.0556	0.2317
M	2	0.0556	0.2317
E, G, I, L, N, S, T, V, Z	1 c/u	0.0278	0.1436 c/u

Comprobación: 9 + 4 + 3 + 3 + 4·2 + 9·1 = 36.

2.2 Entropía de Shannon (inciso B)

H(X) = −Σ p_i·log2(p_i) = 0.5 + 0.3522 + 2(0.2987) + 4(0.2317) + 9(0.1436) = **3.6688 bits/símbolo**.

2.3 Construcción del árbol de Huffman (inciso C)

Bosque inicial (17 hojas, ordenadas por frecuencia): E:1, G:1, I:1, L:1, N:1, S:1, T:1, V:1, Z:1, B:2, C:2, D:2, M:2, R:3, _:3, O:4, A:9.

En cada paso se extraen los dos nodos de menor peso y se unen en un padre con la suma de pesos (empates resueltos por orden de inserción):

Paso	Nodos unidos	Peso nuevo
1	E:1 + G:1	2
2	I:1 + L:1	2
3	N:1 + S:1	2
4	T:1 + V:1	2
5	Z:1 + B:2	3
6	C:2 + D:2	4
7	M:2 + (EG):2	4
8	(IL):2 + (NS):2	4
9	(TV):2 + R:3	5
10	_:3 + (ZB):3	6
11	O:4 + (CD):4	8
12	(M(EG)):4 + ((IL)(NS)):4	8

Paso	Nodos unidos	Peso nuevo
13	((TV)R):5 + (_(ZB)):6	11
14	(O(CD)):8 + ((M(EG))((IL)(NS))):8	16
15	A:9 + (((TV)R)(_(ZB))):11	20
16	(rama de 16) + (rama de 20)	36 (raíz)

Se hacen k − 1 = 16 uniones, como corresponde a un árbol binario completo con 17 hojas. La raíz pesa 36 = N.

2.4 Tabla de códigos, longitud promedio y desigualdad de Shannon (inciso D)

Símbolo	f_i	Código	|c_i|	f_i·|c_i|
A	9	10	2	18
O	4	000	3	12
R	3	1101	4	12
_	3	1110	4	12
B	2	11111	5	10
C	2	0010	4	8
D	2	0011	4	8
M	2	0100	4	8
E	1	01010	5	5
G	1	01011	5	5
I	1	01100	5	5
L	1	01101	5	5
N	1	01110	5	5
S	1	01111	5	5
T	1	11000	5	5
V	1	11001	5	5
Z	1	11110	5	5
Total	36			**133 bits**

Longitud promedio: L = 133/36 = **3.6944 bits/símbolo**.

Desigualdad de Shannon: **3.6688 ⩽ 3.6944 < 4.6688**. Se cumple. La eficiencia es H/L = 99.3 %.

Nota: con otros criterios de desempate los códigos concretos pueden cambiar, pero la suma Σ f_i·|c_i| = 133 bits (y por tanto L) es la misma, porque todos los árboles de Huffman son óptimos.

2.5 Protocolo de serialización (ejercicio 2)

El descompresor no conoce el alfabeto, por lo que el archivo debe incluir la información del árbol.

Opción A: tabla de frecuencias. 256 enteros de 4 bytes: 256 × 4 = **1024 bytes**, sin importar cuántos símbolos existan. El descompresor reconstruye el árbol repitiendo el algoritmo con las mismas reglas de desempate.

Opción B: árbol en preorden. Un bit por nodo (0 = interno, 1 = hoja) y 8 bits del carácter en cada hoja. Para k = 17 hojas hay k − 1 = 16 nodos internos:

- Bits de marca: 16 (internos) + 17 (hojas) = 33
- Bits de caracteres: 17 × 8 = 136
- Total: (k − 1) + k·(1 + 8) = 16 + 153 = **169 bits = 22 bytes** (redondeo hacia arriba).

La opción B es unas 46 veces más pequeña (22 vs 1024 bytes) y su costo crece con k, no con 256. Además no depende de reproducir los desempates del constructor. Por eso se eligió la opción B en la implementación.

Observación sobre este mensaje: los datos codificados ocupan 133 bits y el árbol 169 bits, así que el archivo útil es de 302 bits ≈ 38 bytes, más grande que los 36 bytes originales. Huffman solo compensa su cabecera cuando el archivo es suficientemente grande.

3. Fase 2. Uso de la IA

3.1 Trampa de las cadenas de texto binarias (auditoría de complejidad)

Prompt ingenuo: "Escribe una función en C++ que recorra el archivo de entrada, sustituya cada carácter por su código de Huffman devuelto como `std::string` (ejemplo: "01011"), concatene todas las cadenas en un gran string y luego lo guarde en disco."

Se implementó esa función (`auditoria_trampa_string.cpp`) y se midió con 100 000 caracteres 'A' y un código de 5 bits ("01011").

A. Por qué no comprime. Cada "bit" del código se almacena como un carácter ASCII `'0'` o `'1'`, es decir, un byte completo. Un bit de información ocupa 8 bits de memoria y de disco. Si el código medio es ⌈bits, la salida mide n·⌈bytes en lugar de n·ℓ/8 bytes. Resultado medido:

Concepto	Bytes
Archivo original	100 000
Salida de la trampa (std::string guardado)	500 000

Concepto	Bytes
Salida correcta con bits empaquetados (500 000 / 8)	62 500

La salida es 5 veces más grande que el original (cada carácter de 1 byte pasa a 5 bytes de texto) y exactamente **8 veces** más grande que la representación correcta. En términos de información, el archivo "comprimido" ocupa 8 veces el espacio necesario, es decir, desperdicia 7 de cada 8 bits.

B. Desperdicio de memoria y sobrecosto por realocación. Sea S = n·Γla cantidad de bits de salida.

- Memoria útil: S/8 bytes. Memoria usada: S bytes (mínimo). Desperdicio: S − S/8 = **7S/8 bytes (87.5 %)**.
- `std::string` crece por duplicación de capacidad. Tras la realocación j la capacidad es C_j ≈ C_0·2^j, y cada realocación copia los bytes ya escritos. Número de realocaciones: ⌈log2(S/C_0)⌉. En la prueba se midieron **16 realocaciones** para S = 500 000.
- Bytes copiados en total: C_0 + 2C_0 + 4C_0 + … ⩽ 2S, es decir **O(S)** amortizado, con O(1) amortizado por cada `+=`. La trampa no es cuadrática, pero sí cuesta del orden de 2S bytes copiados adicionales, más una capacidad ociosa de hasta 50 % por encima de S.
- Pico de memoria: durante una realocación conviven el buffer viejo (C) y el nuevo (2C), hasta 3C bytes simultáneos, además de la copia del texto de entrada.
- Con el empaquetado por bits (bloque de `uint8_t` del tamaño final aproximado) el costo es S/8 bytes de escritura, sin copias intermedias.

C. Contra-prompt.

> "La solución anterior almacena cada bit del código como un byte ASCII, por lo que no comprime y consume 8 veces más memoria de la necesaria, además de realocar el `std::string` sin control. Escribe dos clases en C++17: `BitWriter` y `BitReader`. `BitWriter` debe acumular los bits en un buffer `uint8_t` usando desplazamientos (`<<`) y máscaras/OR (`|` , `&`), escribir un byte al disco cada vez que se completen 8 bits, y exponer `write_bit` , `write_byte` , `write_code` y `flush` . `BitReader` debe leer un byte y extraer cada bit con `>>` y `& 1` , exponiendo `read_bit` y `read_byte` . No se permite ningún `std::string` de bits acumulado en memoria, solo el buffer de un byte. Muestra el código completo."

La solución obtenida está en `huffman_compresor.cpp`: `BitWriter::write_bit` hace `buffer = (buffer << 1) | (bit & 1)` y vacía el byte al llegar a 8 bits; `BitReader::read_bit` hace `(buffer >> (bits_remaining - 1)) & 1`. La memoria auxiliar es de un solo byte.

3.2 El bug del último byte (Padding Trap)

Flujo de bits: `10110011 010` (11 bits). El último byte guardado es `01000000`: tres bits reales (`010`) y cinco ceros de relleno. Al leer, el decodificador no puede distinguir los ceros de relleno de bits de un código real.

A y B. Rutina de decodificación ingenua y su fallo. Una rutina ingenua decodifica "hasta que se acabe el archivo" (`while (read_bit(bit)) { ... }`). Con los códigos de la Fase 1, el código de 'O' es `000`: los cinco ceros de relleno bastan para emitir un **carácter espurio 'O'**, y los dos ceros restantes quedan como un prefijo incompleto. Con otro alfabeto, el relleno podría generar uno o más caracteres basura al final del archivo descomprimido, y el archivo recuperado no sería idéntico al original.

C. Corrección. Se eligió la segunda técnica: guardar en la cabecera el número total de caracteres originales como entero de 64 bits (`uint64_t total_chars`). La cabecera del archivo `.huff` queda:

Campo	Tamaño
`total_chars` (uint64_t)	8 bytes
`unique_symbols` (uint16_t)	2 bytes
Árbol serializado en preorden	(k−1) + 9k bits
Datos codificados + padding (flush)	⌈bits/8⌉ bytes

El decompresor ejecuta exactamente `for (i = 0; i < total_chars; ++i)` y decodifica ese número de símbolos; los bits de relleno nunca se interpretan. El campo `unique_symbols` permite detectar el caso de un solo símbolo.

También se consideró la alternativa de guardar el número de bits válidos del último byte (3 bits en el ejemplo, 1 byte en la cabecera). Se descartó porque obliga a calcular cuántos bits totales se escriben y no protege contra otros errores de lectura; el contador de caracteres es más simple y verificable.

4. Caso borde: alfabetos de un solo símbolo

Si la entrada tiene un único símbolo, un árbol con una sola hoja asignaría un código vacío (0 bits). El código lo resuelve con un nodo padre ficticio (`HuffmanNode(single, nullptr)`), de modo que la hoja recibe el código `"0"` (1 bit). `serialize_tree` y `deserialize_tree(reader, is_single_symbol)` manejan ese nodo con un solo hijo, y `generate_codes` asigna `"0"` si el prefijo queda vacío. Un archivo vacío se maneja escribiendo solo la cabecera (`total_chars == 0`).

5. Benchmarking

Compilación: `g++ -O2 -std=c++17 huffman_compresor.cpp` . En todos los casos el archivo descomprimido se verificó byte a byte contra el original (`cmp`).

Ratio de compresión = Bytes comprimidos / Bytes originales × 100 %.

Caso	Original (B)	Comprimido (B)	Ratio	H(X) empírica	Recuperación
A. Texto estructurado (código fuente C++/Python, concatenado)	14 459	8 916	61.66 %	4.8422 bits/byte	Idéntica
B1. Binario aleatorio (1 MB, equivalente a un archivo ya comprimido)	1 000 000	1 000 330	100.03 %	7.9998 bits/byte	Idéntica
B2. Archivo .zip (1 004 495 B)	1 004 495	1 004 825	100.03 %	7.9998 bits/byte	Idéntica
C. 10^7 repeticiones de 'A'	10 000 000	1 250 012	12.50 %	0 bits/byte	Idéntica

A. Texto estructurado. La distribución de caracteres no es uniforme (espacios, minúsculas, signos frecuentes), con H ≈ 4.84 bits/byte frente a 8 bits/byte del ASCII. Huffman ofrece un 61.66 % del tamaño original, cerca del límite H/8 = 60.5 %, con una pequeña diferencia por la cabecera y por usar longitudes enteras. Archivos más grandes y de lenguaje natural (por ejemplo El Quijote) se acercan también a ese límite.

B. Archivos ya comprimidos. Un archivo comprimido o cifrado tiene los 256 valores de byte casi equiprobables, así que H ≈ 8 bits/byte. Por el teorema de la codificación de fuente, ningún código de símbolo a símbolo puede bajar de H bits/símbolo en promedio. El árbol de Huffman resulta casi balanceado (todos los códigos de 8 bits) y el resultado es una **expansión**: 330 bytes más, que corresponden exactamente a la cabecera (10 bytes) más el árbol serializado (256 hojas: 255 + 256·9 = 2559 bits ≈ 320 bytes). Por eso conviene no volver a comprimir con Huffman un .zip, .jpg o .mp4.

C. Archivo homogéneo (10^7 'A'). El programa no falla: no hay división entre cero (la única división es el ratio de tamaños, que solo sería problemática con un archivo de 0 bytes), no hay bucle infinito y no hay desbordamiento de recursión. La profundidad del árbol es 1 (padre ficticio + hoja), y en el peor caso general la recursión está acotada por k − 1 ⩽ 255 niveles. El archivo se recuperó con exactamente 10 000 000 bytes. El resultado es 1 250 012 bytes: 1 bit por carácter (10^7 bits = 1 250 000 bytes) más 12 bytes de cabecera y árbol. Es el mínimo posible para Huffman, que no puede asignar menos de 1 bit por símbolo. Una codificación por longitud de corrida (RLE) lo reduciría a unos pocos bytes.

6. Análisis del código final

- **Comparador determinista:** `NodeComparator` desempata por carácter, así las ejecuciones producen siempre el mismo árbol.
- **Frecuencias con `uint64_t`:** evitan desbordamiento en archivos grandes.
- **Construcción del árbol:** O(k log k) con k ⩽ 256. El tiempo total del compresor es O(n + k log k), dominado por las dos pasadas sobre el archivo.
- **Memoria auxiliar:** O(k) para el árbol y la tabla de códigos, más un byte de buffer de bits. No se guarda el archivo completo en memoria.
- **Dos pasadas:** una para contar frecuencias y otra (tras `seekg(0)`) para escribir los códigos.
- **Verificación automática:** `main` ejecuta el análisis de la Fase 1 (H = 3.6688, L = 3.6944) y la prueba de 10^7 'A', y acepta `-c` / `-d` para comprimir y descomprimir.

7. Conclusiones

1. Para el mensaje M, Huffman produce L = 3.6944 bits/símbolo, a 0.0256 bits de la entropía (3.6688), y cumple H ⩽ L < H + 1.
2. Serializar el árbol en preorden (22 bytes para k = 17) es mucho más económico que guardar una tabla de 256 enteros (1024 bytes). Aun así, con mensajes cortos la cabecera puede superar el ahorro.
3. Guardar los bits como texto (`std::string`) no comprime: ocupa 8 veces más que el empaquetado real y realoca la memoria varias veces. El empaquetado con desplazamientos y máscaras es obligatorio.
4. El relleno del último byte puede generar caracteres basura; guardar `total_chars` en la cabecera lo elimina por completo.
5. Huffman funciona bien con texto y código fuente (61.66 %), expande archivos ya comprimidos (100.03 %) y, en el caso extremo de un solo símbolo, queda limitado a 1 bit por símbolo (12.50 %).

6. Las IAs generan con facilidad soluciones que "funcionan" pero fallan en complejidad o en casos borde; hay que auditarlas con pruebas (aquí: tamaño de salida, realocaciones, último byte y archivo homogéneo).

8. Anexos

- Archivos entregados: `huffman_compresor.cpp`, `auditoria_trampa_string.cpp`, `fase1_verificacion.py`.
- Conversación con la herramienta de IA (requerida por la práctica): [pegar aquí el enlace o la transcripción completa de la conversación, incluyendo el prompt ingenuo, el contra-prompt del BitWriter/BitReader y el prompt de corrección del último byte].
- Nota sobre la Fase 1: la teoría se resolvió sin IA, y el script `fase1_verificacion.py` se usó solo para verificar los resultados.